

挑战三

实验步骤:

下载并校验课程负载:

```
(ray-serve) measure@LAPTOP-CMJ:~/26fall-HW-data/workloads/hw2/target3-routing-policies$ python validate_workload.py moon
cake_prefix_workload_v2_seed2026.jsonl
{
  "sha256": "f154ba954e28ca9612257edf5deb0d5dfc6bf27fd0115a701e1f49e52df75b03",
  "records": 2112,
  "families": 64,
  "warmups": 64,
  "measured": 2048,
  "scheduled": true,
  "planned_duration_s": 29.538768,
  "tier_families": {
    "cold": 32,
    "hot": 7,
    "superhot": 1,
    "warm": 24
  },
  "tier_requests": {
    "cold": 240,
    "hot": 560,
    "superhot": 768,
    "warm": 480
  },
  "traffic_phases": {
    "burst": 1024,
    "recovery": 512,
    "steady": 512
  },
  "affinity_family_distribution": {
    "0": 16,
    "1": 16,
    "2": 16,
    "3": 16
  },
  "affinity_request_distribution": {
    "0": 1027,
    "1": 339,
    "2": 342,
    "3": 340
  }
}
```

根据题目要求检验负载

搭建四副本服务并完成四组对照:

通过 AutoDL 租一台 GPU:RTX 3090 24GB(详细数据见 README), 分四块用, 在同一台服务器上搭建 4 个配置完全相同的 SGLang 服务, 用 ray.cluster_utils.Cluster 起 1 个 Ray head 节点+4 个逻辑 worker 节点, 部署 4 个 Ray Serve Replica,每个 Replica 独占一个 worker 节点并固定转发到其中一个 SGLang 后端

用同一份固定负载比较 5 种路由配置:

组	路由类	max_ongoing_requests	说明
A	P2C	5	默认请求路由器
B1	P2C	8	候选 1, 非默认值
B2	P2C	32	候选 2, 非默认值
C	ConsistentHashRouter	32	纯前缀亲和
	Num_fallback_replicas=0		
D	PrefixAffinityLoadAwareRouter	32	前缀亲和和优先 负载感知让位

运行环境和软件版本以及详细执行步骤以 README 为主

五轮严格执行同一套动作：

- 1.起 Ray 集群（1 head + 4 worker）→部署 Serve 应用→verify_placement()自检；
- 2.对 4 个后端分别 POST /flush_cache，把上一轮残留的 KV 全部清掉，保证每轮都从冷缓存开始；
3. 在另一个终端跑 run_workload.py（64 预热 + 2048 测量）；
4. 收工拆除集群（serve.shutdown()→ray.shutdown()→cluster.shutdown()），下一轮重新建。每轮重建集群意味着每轮都是冷启动，四轮起点一致

分别开启 A-D 的 RayServe 并分别运行 run_workload.py 得到数据结果

结果包含

文件	内容
config.json	命令、模型、路由器、并发参数、负载 SHA-256、校验状态和自定义备注
warmups.csv	64 条预热请求的结果
requests.csv	2048 条测量请求的逐请求结果
summary.json	全程、各流量阶段和各热度层级的汇总指标

详细字段含义见 README

主表列出 A、两个 B 候选、C 和 D 的成功率、吞吐量、缓存命中率、实际 Prefill token 数、TTFTp95、端到端延迟 p95，以及四个 SGLang 后端的请求分布

主表：

组别	A	B1	B2	C	D
成功率	100%	100%	100%	100%	100%
吞吐量	6.169	9.109	17.755	13.202	15.859
缓存命中率	0.5811	0.5806	0.5831	0.8332	0.6001
实际 Prefill token 数	1,354,496	1,354,496	1,348,096	539,263	1,292,864
TTFTp95	284.2	183.9	78.3	115.5	94.7
端到端延迟 p95	286.6	187.4	83.0	119.2	99.6

后端分布请求(30000/30001/30002/30003)：

A:510/493/526/519

B1:517/495/532/504

B2:522/525/492/509

C:1012/478/279/279

D:461/529/514/544

分析结果：

根据主表的直观数据来看：

- 1. A 的 TTFT p95=284.2s,比其他任何组都高一个数量级，说明在 A 的配置下，绝大多数请求是从到达起就一直在排队，一直排到整轮快结束才被服务
- 2. C 用 60.2%的 prefill 换来了 3.4 倍的分布倾斜，它的实际 prefill token 只有 A 的 39.8%，但是后端分布从 A 的 33 极差放大到了 733 极差
- 3. D 和 B2 的指标几乎都在同一量级，但成因不同，B2 靠加大 max_ongoing_request，D 靠前缀复用+更闲的副本

根据 comparison.json 列出相对基准 A 的变化：

组	吞吐提升	TTFT p95 降低	端到端 p95 降低	缓存命中率变化
B1	+47.7%	-35.3%	-34.6%	-0.0005
B2	+187.8%	-72.4%	-71.0%	+0.0020
C	+114.0%	-59.4%	-58.4%	+0.2521
D	+157.1%	-66.7%	-65.2%	+0.0191

结合数据回答：

Q1: 默认值与 B 中选定值为何表现不同

差异来自 max_ongoing_requests，B2>B1>A，从而使得每个副本上允许同时存在的请求上限增加，从而提升整体吞吐量

组	设定值	副本总上限	吞吐量	整轮耗时	端到端 p50	端到端 p95
A	5	20	6.169	331.98	151.5s	286.6s
B1	8	32	9.109	224.82	97.4s	187.4s
B2	32	128	17.755	115.355	43.3s	83.0s

max_ongoing_requests 的增加，使得每个 SGLang 后端收到更多的请求，减少请求进批的数量和 GPU 花在这上面的时间，单位时间算完的 token 更多，但单请求的流式输出变得更慢，即用单请求的逐 token 延迟换整体吞吐与排队时间
所以 C 组选择 32，三项主要指标都有数量级优势，而缓存命中率与后端分布几乎不动

Q2: 前缀亲和为什么同时带来缓存收益和热点

两者出自同一个约束。一致性哈希给出的路由函数 prefix_family → backend 是单值且与请求时刻无关的映射，其形式化后果有二：其一，同一族的全部请求持续到达同一副本，该副本的前缀 KV 常驻集收敛于该族模板，前缀命中率上升

(收益)；其二，族请求量的分布被一对一传递为副本请求计数的分布。本负载的族请求量分布高度重尾，故该约束的代价被放大到最大。

前缀亲和性之所以同时带来缓存收益和热点，是因为它本质上是在做一种局部性优化，也就是把相同前缀的请求集中到同一个副本上。这既让缓存命中的收益最大化，也必然导致流量向少数“热”副本聚集。两者是同一机制的一体两面。

根据数据可知：C 的缓存命中率升到 0.8332，实际 prefill 相对于 A 降到了 39.8%

组	缓存命中率	实际 Prefill token
A	0.5811	1,354,496
C	0.8332	539,263

完全亲和模型把每族全部请求落在同一副本，首次之后每次都命中整段族模板。根据数据分析可知，C 组的 superhot 档合规率是 1.00，亲和合规率只有 C 达到 0.6128。前缀亲和路由的作用，就是把带有相同前缀的请求“粘”到缓存了该前缀的副本上：

请求不再触发完整的 Prefill，而是直接复用已缓存的 KV 状态，只需处理新增的 Token。这能把昂贵的 Prefill 计算转化为一次近乎免费的缓存查找，显著降低 TTFT。

同时也带来热点问题，路由决策不再以负载均衡为首要目标，如果某个副本缓存了一个高频共享前缀，那么所有携带该前缀的请求都会被路由到这一个副本上。随着并发请求涌入，这个“热”副本的队列会迅速堆积，而其他缓存了不同前缀的副本却可能处于空闲状态。affinity 把流量集中到了 cache-warm 的端点上，如果没有显式的“饱和释放”机制，热端点会持续吸收负载，直到超过冷端点本可以更快服务的阈值。

Q3: D 使用了哪些信号与规则，改善了什么，又付出了什么代价

D 组的自定义路由类 PrefixAffinityLoadAwareRouter(继承 ConsistentHashRouter)，详细看源码 router_d.py

使用信号：

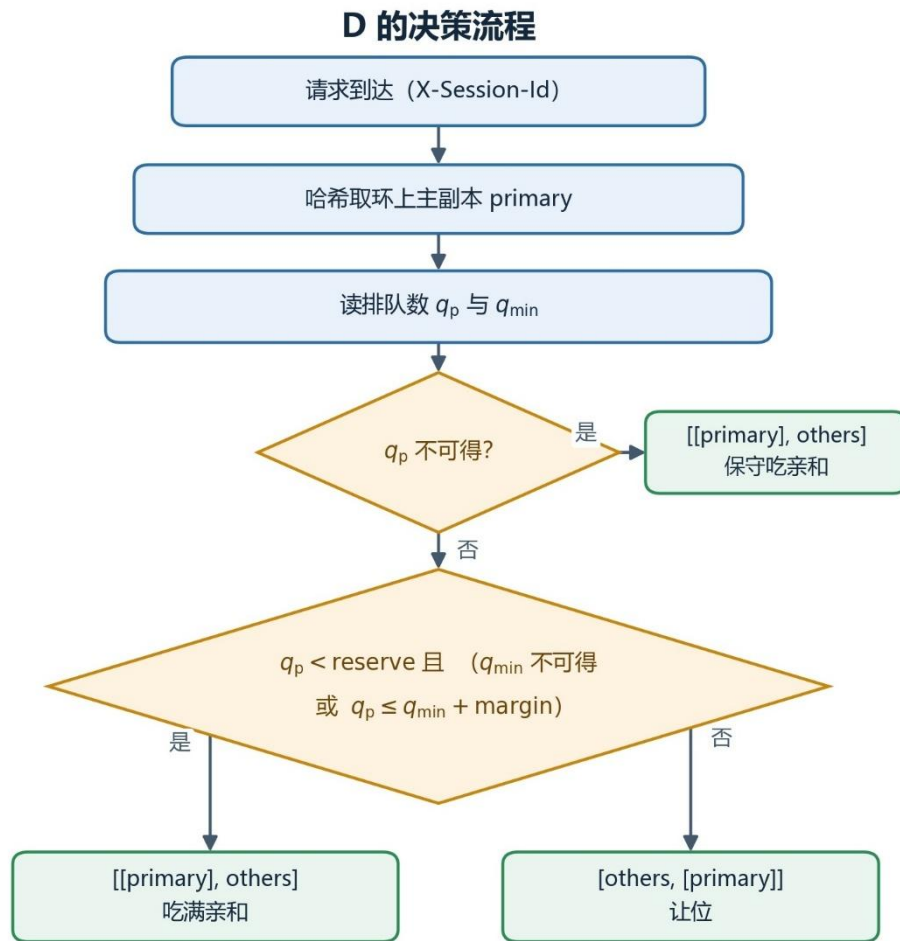
信号	来源
前缀亲和主副本	ConsistentHashRouter._lookup_ranked_replicas(key)
副本排队长度	RequestRouter._replica_queue_len_cache.get(replica_id)

规则：

Ray Serve 的 choose_replicas()返回的是候选名次表 List[List[RunningReplica]]:

外层是名次优先，内层由框架自动挑排队最短的哪个。

D 的决策流程：



改善：

相对 A

指标	A	D	变化
吞吐量	6.169	15.859	+157.1%
TTFT p95	284.2s	94.7s	-66.7%
端到端 p95	286.6s	99.6s	-65.2%
缓存命中率	0.5811	0.6001	+0.0191
实际 Prefill	1,354,496	1,292,864	-4.5%

D 是五组里唯一“既拿到部分缓存收益、又没有产生热点”的配置

付出代价：

代价一：TPOT 五组最差。D 的 TPOT 均值 0.0474s，比 B2 的 0.0427 高 11.0%，是 A 的两倍。批越满，单请求每 token 越慢 D 的派发让每个副本都更“守恒”地保持满载，代价落在单请求的流式输出上。

代价二：路由决策带来的调度滞后可测量。D 的 dispatch_lag p95=0.0197s，

是其他四组 (0.0013 - 0.0016 s) 的 12 - 15 倍。绝对值仍然极小,但它是“自定义路由在代理进程里多做一层判断”的直接痕迹。

代价三: 亲和分支在本负载下几乎没生效。37632 次决策里只有 8 次走了亲和分支。原因是结构性的: 本负载把系统压到 4 倍过载, 四个副本的排队数同时顶在 `max_ongoing_requests=32` 这个饱和值上, 于是 $q_p < 16$ 恒假, $q_p \leq q_{min} + 4$ 也恒假——两个判据都失去了区分度, 路由退化成“总是让位给其余副本里最闲的那个”。D 整体只有 0.2378、在 `superhot` 档只有 0.1888, 都低于随机的 0.25——日志说“D 只走了 8 次亲和”,

代价四: 分布反而不如默认的 P2C 均匀。D 的 `imbalance` 0.1621 > B2 的 0.0645。

这与代价三是同一个根因: 饱和时排队长度信号不再有区分度, “挑最闲”退化为在陈旧读数上的

平局裁决, 反而不如 P2C “随机抽两个比一比”来得均匀。

代价五: 相对 B2 并不占优。也就是说, D 相对 B2 是“用 10.7% 的吞吐换 2.9% 的命中率”, 在同一条折中曲线上并不优于 B2。结合代价三、代价四, 可以确认: D 拿到的这点命中率提升不是来自它设计的“前缀亲和”, 而是来自“比 P2C 更细粒度的最闲副本挑选”。这一条现在有逐请求的量化证据: D 的族内一致性 0.3296 与 B2 的 0.3394 基本相同, 说明 D 并没有比 B2 更强地将同一族约束在同一副本上; D 唯一明显高于 B2 的量是同族相邻请求落同一后端的比例。

作业感受:

1. 简述下你是如何完成第三次挑战的。(例如, 如何使用 AI、如何工作迭代等方面)

本次挑战我依旧使用了前几次的 ai, 工作迭代: 先在本地克隆了仓库, 提前校验了负载结果, 然后在 AutoDL 租一个 RTX 3090, 配置按照 claude 给出的最佳性价比方案, 在新 GPU 上 conada 了两个虚拟环境, 部署 Qwen3.08B, 克隆仓库, 一个用来启动 SGLang 的四个服务端, 一个用来启动 Ray Serve。配置了相应的环境和软件。让 claude 帮我生成 `app.py` 脚本, 包含了 Ray 集群与 Serve Deployment, 以及 `router_d.py` 脚本——D 的路由代码, 详细复现过程请看 README。接着启动测试 A-D, 最后写 README 和 report

2. 你觉得哪部分工作最困难, 你是如何克服这个困难的。

主要是在新的 GPU 上要重新配置环境版本有时候不对, 要重新配置挺麻烦的让 ai 帮我检查版本, 我再更新或降低版本。以及理解一下新概念有些名词比较难理解, 通过询问 ai 和文档来逐步理解

3. 你觉得类似的从 0 到 1 的科研工作, 需要包含哪些方面的研究, 对你今后的科研工作有什么启发

需要学会测量数据，通过不断的对比和优化方法来得到较优方案，学会了利用 AutoDL 来租 GPU 跑深度学习，神经网络等